Datalog: Logic and databases

1 Datalog, the theoretical point of view.

There are several ways to look at Datalog programs. Often seen as database systems, a datalog program will, in this lab, be seen as a set $\mathcal{R}$ of logical rules over a set of predicates $P_1 \dots P_k$.

Each rule is of the form $\forall \vec{x}\,(P_1(\vec{w}_1) \wedge \cdots \wedge P_k(\vec{w}_k)) \Rightarrow P(\vec{y})$ where P and the P_i are predicates. The $\vec{x}$, $\vec{y}$ and the $\vec{w}_i$ are made of variables or constants but we further impose that all variables in $\vec{y}$ have to appear in at least one of the $\vec{w}_i$ and, in turn, all variables in the $\vec{w}_i$ have to appear in $\vec{x}$ (the rule is a closed formula).

An exemple. Let us consider the three following rules over the constants `Socrates`, `Human`, `Mortal`:

- `is(Socrates,Human)` (here the quantification $\forall$ disappear as we quantify over an empty set of variable)
- `is(Human, Mortal)`
- $\forall x, y, z$ $\texttt{is}(x, y) \wedge \texttt{is}(y, z) \Rightarrow \texttt{is}(x, z)$.

Models. A *model* $\mathcal{I}$ of the set of rules $\mathcal{R}$ is any interpretation of the predicates such that all rules are satisfied, i.e. for each $\varphi \in \mathcal{R}$ we have that $\mathcal{I} \vDash \varphi$.

Follow up on the exemple. There are many models for this set of rules but one can easily see that in any model, the interpretation of the predicate `is` will contain: `is(Socrates,Human)`, `is(Human, Mortal)`, `is(Socrates, Mortal)` as otherwise at least one of the three rules will not be satisfied.

Remark about existential quantification: The rules in $\mathcal{R}$ are all of the form $\forall \vec{x}\,(P_1(\vec{w}_1) \wedge \cdots \wedge P_k(\vec{w}_k)) \Rightarrow P(\vec{y})$ but we can restrict $\vec{x}$ to be exactly the set of free variables in $(P_1(\vec{w}_1) \wedge \cdots \wedge P_k(\vec{w}_k)) \Rightarrow P(\vec{y})$. Therefore, we usually omit writing down the universal quantification and write rules as $(P_1(\vec{w}_1) \wedge \cdots \wedge P_k(\vec{w}_k)) \Rightarrow P(\vec{y})$.

Remark about fact VS rules: Rules in $\mathcal{R}$ can be of the form $\top \Rightarrow P(\vec{y})$ where $\vec{y}$ can only be composed of constants. In that case we say that such a rule is a *fact*. Rules that are not facts are said to be *deduction rules*.

Follow up on the example. In our example, there were twoq facts `is(Socrates,Human)` and `is(Human, Mortal)` and one deduction rule $\forall x, y, z$ $\texttt{is}(x, y) \wedge \texttt{is}(y, z) \Rightarrow \texttt{is}(x, z)$.

Minimal model. Datalog enjoys what is called at the minimal model property. For any set of rules $\mathcal{R}$ there exists a model $\mathcal{M}$ such that any model $\mathcal{M}'$ of $\mathcal{R}$ is such that any fact that holds in $\mathcal{M}$ also holds in $\mathcal{M}'$.

Negation. In pure Datalog, you **cannot** express negation. There are various extensions and some support negation but then you loose the minimal model property. In this lab we will not be using those extensions and thus you don't have negation.

Evaluation of a datalog program. Given a datalog program, i.e. a set of rules $\mathcal{R}$, the evaluation of the program is to return the minimal model. For optimization and practical purposes, datalog evaluators often do not return the full model but rather we ask a question of the form $P_1(\vec{w}_1) \wedge \cdots \wedge P_k(\vec{w}_k)$ and the evaluator returns the list of substitution that makes this formula true in the minimal model.

2 Practical Datalog.

Datalog is real database language that you can manipulate. To practice it you can go on `https://jachiet.com/tmp/dl/`.

Here are some elements of syntax:

- to distinguish between variables and constants, datalog uses the case: uppercase is variable and lowercase is constant;
- a fact $P(c_1, \ldots, c_k)$ is written `P(c1, ..., ck).` (the final dot is important here);
- a rule such as $\forall x, y, z S(x, y) \wedge T(y, z) \wedge S(y, \textit{bob}) \Rightarrow P(x, z, \textit{alice})$ is written as `P(X,Z,alice) :- S(X,Y), T(Y,Z), S(Y, bob).` Notice the mix of constants (Alice and Bob) and variables.
- to retrieve the solutions of a query that is a single predicate $R(x, \textit{charlie})$, i.e. all the assignations of the variable x such that $R(x, \textit{charlie})$, we can write `R(X,charlie)?` (note the final interrogation point that replaces the dot). Datalog will then output all R facts that can be obtained by substituting the variable x.

Exercise 1. Load the scenario "cousin" on the website and find one (or several) rules to define cousin such that cousin(x,y) is true when x and y share a grand parent. Yes: this definition allows one person to be its own cousin or cousin with a sibling (but since Datalog does not allow negation we could not prevent someone from being its own cousin). The predicate parent(x,y) means that y is the parent of x.

Exercise 2. Load the scenario "edge" on the website and define a predicate such that path(x,y) holds when there is a path from x to y (you can consider the graph oriented and say that edge(x,y) holds when $x \to y$).

Exercise 3. Load the scenario "edge" on the website, what are the vertices that belong to a cycle?

Exercise 4. Load the scenario "edge" on the website, what are the vertices that belong to a cycle of odd length?

Hard questions: arithmetic

Load the scenario Peano, in this scenario we have the relation "succ" that is defined such that "succ(x, y)" holds when $y = x + 1$. For finiteness reasons, we only define succ up to 29.

Exercise 5. We want to define a predicate "padd" such that "padd(x, y, z)" holds when $x + y = z$. Using the Peano addition, we know that "padd" can entirely be defined through the rules $(a + 1) + b = (a + b) + 1$ and $a + 0 = a$. Check your result (e.g. 3+4=?, 7+5=?). Because "succ" is only defined up to 30, your predicate will only be defined for $0 \leq x, y, z \leq 30$. Feel free to check `https://en.wikipedia.org/wiki/Peano_axioms#Addition`.

Exercise 6. Using the predicate "padd", can you define a "pminus(x, y, z)" that holds when $x - y = z$? Your predicate will only be defined for $0 \leq x, y, z \leq 30$.

Exercise 7. Can you define a "pmult(x, y, z)" that holds when $x * y = z$? Your predicate will only be defined for $0 \leq x, y, z \leq 30$. Here the important facts are $(x + 1) \times y = x \times y + y$ and $0 \times y = 0$. Feel free to check `https://en.wikipedia.org/wiki/Peano_axioms#Multiplication`.

Very hard questions

The following exercise is particularly hard, don't spend too much time on it.

Exercise 8. Rule 110 is a 1D cellular automaton, you can look on wikipedia to get a better understanding of how it works `https://en.wikipedia.org/wiki/Rule_110`. Define rules for tape so that `tape(X,Y,Z)` holds when the tape contains `Z` after `X` steps in the cell `Y`. The cellular automaton is on a infinite 1D tape which we cannot simulate in Datalog, in fact the border of the tape will disappear (initially the tape is defined between position 0 and 30 and after X steps the tapes the tape will be defined at positions between X and 30-X.